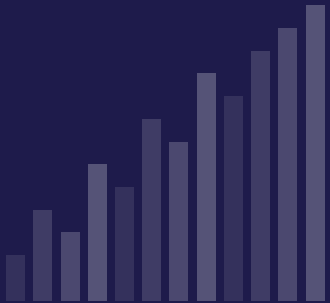
AULA 16 · BLOCO 2: ORDENAÇÃO AVANÇADA E ESTRUTURAS LINEARES

Pilhas, filas e vetores dinâmicos

Cormen et al., *Algoritmos*: seção 10.1 e seção 17.4



TEORIA

Roteiro da aula

Ao final, você será capaz de

- Implementar pilha (LIFO) e fila circular (FIFO) com vetores
- Tratar transbordamento e esvaziamento
- Analisar o vetor que dobra de tamanho: custo amortizado $O(1)$
- Medir $2n$ cópias contra $n^2/2000$ para duas estratégias de crescimento

Teoria

Pilha, fila circular, análise amortizada da tabela dinâmica

Prática em C

Pilha dinâmica e fila circular em C; parênteses balanceados

Experimento

Dobrar ou somar 1000? Contando cópias

Conexões

Compiladores, navegadores, sistemas operacionais e linguagens



Teoria

- Cormen et al., seção 10.1 e seção 17.4
- Pilha: último a entrar, primeiro a sair
- Fila circular: primeiro a entrar, primeiro a sair
- E quando o vetor enche? Tabela dinâmica

TEORIA

Pilha: último a entrar, primeiro a sair

PUSH(S, x): $S.topo \leftarrow S.topo + 1$; $S[S.topo] \leftarrow x$. POP(S): se vazia, **erro**; senão devolve $S[S.topo]$ e decrementa o topo. Todas $O(1)$.

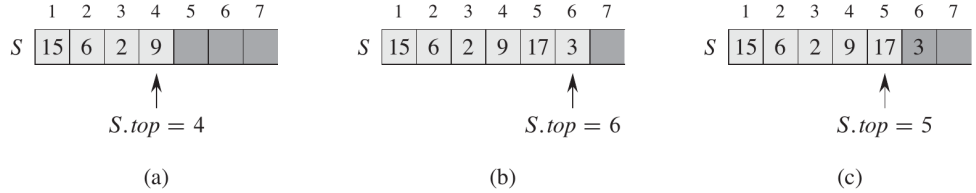


Figura 10.1 de Cormen et al. (2009) — (a) pilha com 4 elementos; (b) depois de PUSH(17) e PUSH(3); (c) depois de POP.

TEORIA

Fila circular: primeiro a entrar, primeiro a sair

- *cabeça*: próximo a sair. *cauda*: próxima posição livre.
- Os índices andam “em círculo”: depois de n vem 1. Em C: `(i + 1) % n`.
- Vazia: *cabeça* = *cauda*. Para distinguir cheia de vazia, guarde o número de elementos (ou deixe uma posição livre).

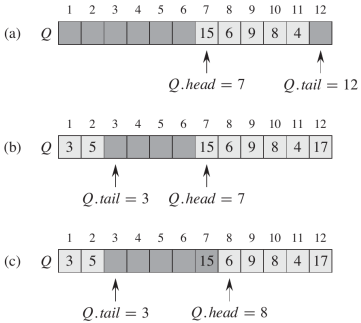


Figura 10.2 de Cormen et al. (2009) — (a) fila com 5 elementos; (b) após Enqueue de 17, 3 e 5: a cauda dá a volta; (c) após Dequeue.

TEORIA

E quando o vetor enche? Tabela dinâmica

Ao encher, aloque o **dobro** e copie tudo.

- Uma inserção pode custar $\Theta(n)$ (a cópia).
- Mas n inserções custam no total $n + (1 + 2 + 4 + \ldots) < 3n$.
- **Custo amortizado** $O(1)$ por inserção.
- Somar uma constante c em vez de dobrar: total $\Theta(n^2/c)$.

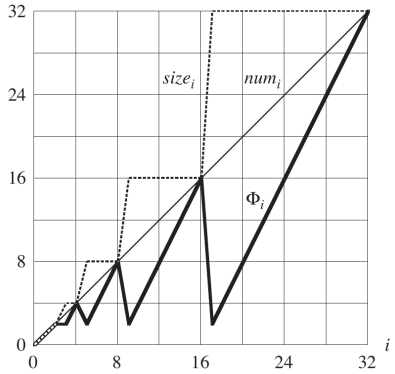


Figura 17.3 de Cormen et al. (2009) — tamanho alocado ($size_i$), ocupação (num_i) e potencial Φ_i ao longo de 32

Prática em C

- Pilha que dobra com `realloc`
- Fila circular de tamanho fixo (Figura 10.2)
- Aplicação: parênteses, colchetes e chaves balanceados

PRÁTICA EM C

Pilha e fila em C (1/4)

```c
#include <stdio.h>
#include <stdlib.h>

/* Pilha que dobra a capacidade quando enche: PUSH custa O(1) amortizado. */
typedef struct { char *v; int topo, cap; } Pilha;

void empilha(Pilha *p, char c) {
    if (p->topo == p->cap) {
        p->cap = p->cap ? 2 * p->cap : 4;
        p->v = realloc(p->v, p->cap);
    }
    p->v[p->topo++] = c;
}
int vazia(const Pilha *p) { return p->topo == 0; }
char desempilha(Pilha *p) { return p->v[--p->topo]; }   // supõe não vazia
```

PRÁTICA EM C

Pilha e fila em C (2/4)

```c
/* Fila circular de tamanho fixo (Figura 10.2): cabeca e cauda dão a volta. */
#define TAM_FILA 8
typedef struct { int v[TAM_FILA]; int cabeca, cauda, n; } Fila;

int enfileira(Fila *f, int x) {
    if (f->n == TAM_FILA) return 0;                    // transbordamento
    f->v[f->cauda] = x;
    f->cauda = (f->cauda + 1) % TAM_FILA;
    f->n++;
    return 1;
}
int desenfileira(Fila *f) {                            // supõe não vazia
    int x = f->v[f->cabeca];
    f->cabeca = (f->cabeca + 1) % TAM_FILA;
    f->n--;
```

PRÁTICA EM C

Pilha e fila em C (3/4)

```
    return x;
}

int balanceada(const char *s) {                          // aplicação clássica da pilha
    Pilha p = {NULL, 0, 0};
    int ok = 1;
    for (int i = 0; s[i] && ok; i++) {
        char c = s[i];
        if (c == '(' || c == '[' || c == '{') empilha(&p, c);
        else if (c == ')' || c == ']' || c == '}') {
            char esperado = c == ')' ? '(' : c == ']' ? '[' : '{';
            ok = !vazia(&p) && desempilha(&p) == esperado;
        }
    }
    ok = ok && vazia(&p);
```

PRÁTICA EM C

Pilha e fila em C (4/4)

```
    free(p.v);
    return ok;
}

int main(void) {
    char s[1001];
    if (scanf("%1000s", s) != 1) return 1;
    printf(balanceada(s) ? "balanceada\n" : "não balanceada\n");
    Fila f = {{0}, 0, 0, 0};                          // demonstração da fila circular
    for (int x = 1; x <= 10; x++)
        if (!enfileira(&f, x)) desenfileira(&f), enfileira(&f, x);
    printf("fila (mais antigo primeiro):");
    while (f.n) printf(" %d", desenfileira(&f));
    printf("\n");
    return 0;
}
```



Experimento

- Dobrar: $\approx 2n$ cópias (amortizado $O(1)$)
- Somar 1000: $\approx n^2/2000$ cópias
- $n = 2^k + 1$: logo após uma expansão

EXPERIMENTO

Dobrar ou somar 1000?

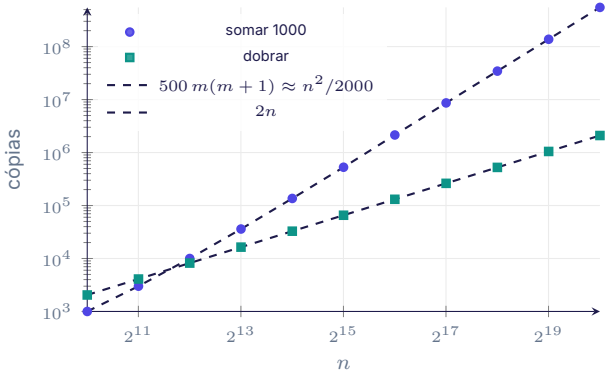
Hipótese. Dobrando, as cópias são $1+2+4+\cdots+2^k = 2^{k+1}-1 \approx 2n$ quando $n = 2^k + 1$: amortizado, **2 cópias por inserção**. Somando 1000 posições, há $m = \lfloor (n-1)/1000 \rfloor$ expansões que copiam $1000, 2000, \ldots, 1000m$: total $500\,m(m+1) \approx n^2/2000$.

Método

1. Simulação da capacidade e das cópias (sem alocar de verdade)
2. $n = 2^{10}+1, \ldots, 2^{20}+1$
3. Capacidade inicial: 1 (dobra) e 1000 (soma)
4. Contagem exata, sem ruído

EXPERIMENTO

Cópias para n inserções



n	medido	medido $\div\, 500\,n$
1.025	1.000	
4.097	10.000	
16.385	136.000	
65.537	2.145.000	
262.145	34.453.000	
1.048.577	549.676.000	

✓ desvio máx. 0,0%

Para um milhão de inserções: 2 milhões de cópias dobrando, 550 milhões somando 1000.

EXPERIMENTO

O que o experimento mostrou

Dobrar custa $\approx 2n$ cópias: **amortizado** $O(1)$ por inserção. Somar uma constante custa $\approx n^2/2000$: quadrático, por maior que seja a constante. Por isso `std::vector`, `ArrayList` e as listas do Python crescem por um **fator** (1,5 ou 2), nunca por uma quantidade fixa.

Reproduza: `gcc -O2 aula16_experimento.c -o exp -lm && ./exp`



Conexões

- Compiladores
- Navegadores e editores
- Sistemas operacionais
- Linguagens

CONEXÕES

Onde isso aparece na computação

Compiladores

Analisadores sintáticos usam pilhas para casar parênteses e blocos; a pilha de chamadas guarda variáveis locais e endereços de retorno.

Navegadores e editores

Voltar e avançar, desfazer e refazer: duas pilhas. O DOM é percorrido com pilha (profundidade) ou fila (largura).

Sistemas operacionais

Buffers de teclado, de rede e de impressão são filas circulares; o *ring buffer* é a estrutura dos drivers de rede de alto desempenho.

Linguagens

A lista do Python, o `vector` do C++ e o `slice` do Go são vetores dinâmicos com crescimento por fator, exatamente a análise desta aula.

PARA CASA

Para casa

Exercícios

1. (10.1-1) Ilustre Push(4), Push(1), Push(3), Pop, Push(8), Pop numa pilha vazia.
2. (10.1-4) Reescreva Enqueue e Dequeue detectando transbordamento e esvaziamento.
3. (10.1-6) Implemente uma fila com duas pilhas. Qual o custo amortizado?
4. E se, além de crescer, o vetor encolher pela metade quando ficar com $1/4$ ocupado? Por que não encolher já com $1/2$ (seção 17.4.2)?
5. Altere o experimento para crescer por fator 1,5 e compare com o fator 2.

Leitura
Cormen et al., seções 10.1 e 17.4 (tabelas dinâmicas).

Código
josedihego.net/estruturas